Session 2: Working like a computational biologist in Python

KB8029 — Bioinformatics & Deep Learning

Arne Elofsson

SciLifeLab

SciLifeLab

Log in to the lecture quiz

Log in to the quiz

Scan the code, or go to quiz.bioinfo.se/live

Log in with your university account

Answering counts as being at the lecture: up to 3 bonus points on the exam

Right or wrong is never graded, only whether you answered. Results are shown without names.

quiz.bioinfo.se/live

SciLifeLab

Warm-up: what do you know already?

3 quick questions

On the topics of today, one at a time, about 30 seconds each

Answer on your own, and guess if you do not know

It shows me where to start; right or wrong is never graded

SciLifeLab

x = [1, 2]; y = x; y.append(3). What is x now?

A [1, 2]

B [1, 2, 3]

C [3]

D An error

Answer now
Scan the code, or go to
quiz.bioinfo.se/live

SciLifeLab

df has one row per gene. What does df["padj"] < 0.05 give?

A The genes with padj below 0.05

B The number of such genes

C True or False for every gene

D An error

Answer now
Scan the code, or go to
quiz.bioinfo.se/live

SciLifeLab

Python reports an error on line 12. Where is the mistake?

A On line 12

B On line 12 or earlier

C After line 12

D In a library, not in your code

Answer now
Scan the code, or go to
quiz.bioinfo.se/live

SciLifeLab

What the room knows already

Where we start

Which topics most of you know, and which we take slowly today.

Show the results ▸ quiz.bioinfo.se/live/report

Your questions before today

What you asked

The questions you sent in on Canvas before the lecture

The most common ones now; the others in the block where they belong

SciLifeLab

In up to three words: what do you want to learn today?

Represent and explore	Questions and tools	Getting unstuck
Seq, DataFrame, class	plot first, SciPy, NCBI	errors, docs, git

Answer now

Scan the code, or go to
quiz.bioinfo.se/live

SciLifeLab

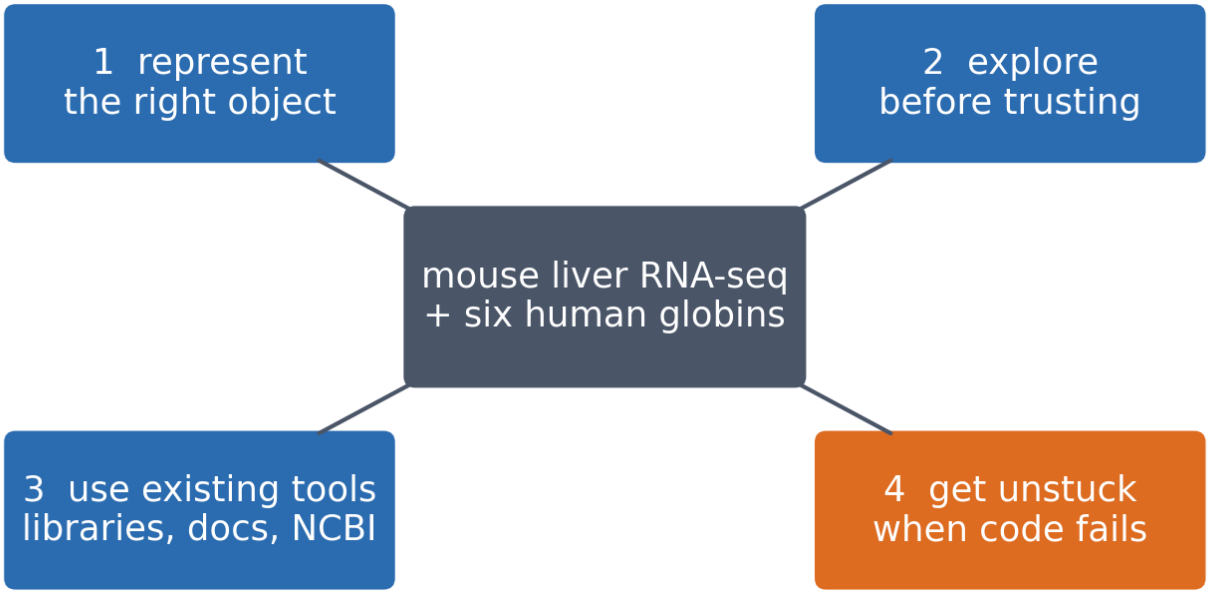
What you want to learn today

What the room wants to learn

Show the results ▸ quiz.bioinfo.se/live/report

SciLifeLab

Four things a computational biologist does with Python



One real dataset (mouse liver RNA-seq, Becares et al. 2019) and one gene family (the human globins) carry all four ideas through the session.

See course.bioinfo.se/session02.html.

SciLifeLab

Three blocks: about 30 min talk and 15 min discussion



Block 1: objects, your own classes, sequences and a first look at the RNA-seq table. Block 2: plot first, test second; reading data; live coding with six globins. Block 3: getting unstuck, git, and the lab.

Times are approximate: a 10 min start (warm-up quiz and your questions), two 15 min breaks and a 5 min close. Each block ends with a quiz question: guess, run the notebook and work out why it behaves as it does, discuss, answer again.

SciLifeLab

BLOCK 1 · ABOUT 30 MIN

Represent and explore

SciLifeLab

The right object for the data

list	`[628, 577, 576]`	any collection
NumPy array	`np.array(lengths)`	numbers, with maths
pandas DataFrame	`pd.DataFrame({...})`	a table, named columns
Biopython Seq / SeqRecord	`Seq('ATGGTG...')`	a sequence that knows biology
PyTorch tensor	`torch.tensor(...)`	an array with gradients (Session 7+)

The same three transcript lengths in increasingly specialised objects. Each object gives you the operations that make sense for its kind of data.

See course.bioinfo.se/session02.html, 'Scientific Python objects'.

SciLifeLab

Same numbers, different object, different answer

```
lengths = [628, 577, 576]
lengths * 2
[628, 577, 576, 628, 577, 576]   repeats the list!
```

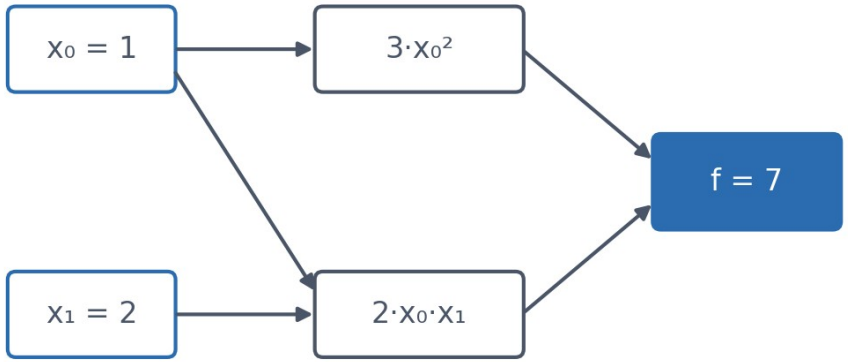
```
arr = np.array(lengths)
arr * 2
[1256 1154 1152]   doubles every number
```

- **A list repeats**
- **An array does the maths**
- Both run without an error

See course.bioinfo.se/session02.html, 'Scientific Python objects'.

SciLifeLab

A tensor remembers how it was computed



f.backward()
chain rule,
back through
the graph

$x_0.\text{grad} = 6 \cdot x_0 + 2 \cdot x_1 = 10$ $x_1.\text{grad} = 2 \cdot x_0 = 2$

- **f.backward(): the chain rule, automatically**
- Session 1's answer: (10, 2)
- From Session 7: no derivatives by hand

See course.bioinfo.se/session02.html, 'Looking ahead: tensors and autograd'.

SciLifeLab

A Seq knows biology

```
hbb = Seq('ATGGTGCATCTGACTCCTGAGGAGAAGTCT')

len(hbb)                    →  30

gc_fraction(hbb)            →  0.5

hbb.reverse_complement()    →  AGACTTCTCCTCAGGAGTCAGATGCACCAT

hbb.translate()             →  MVHLTPEEKS
```

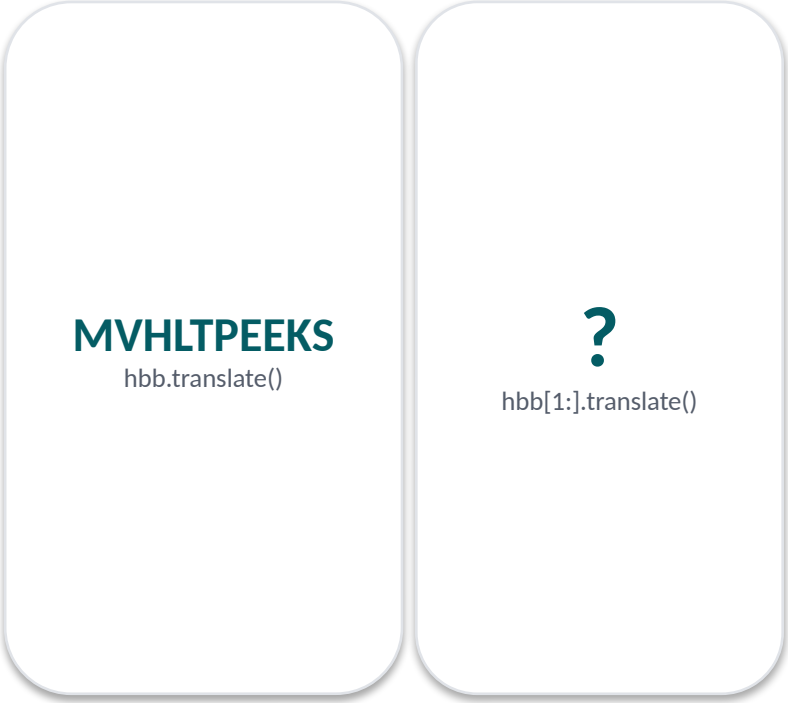
- **The first ten codons of HBB**
- No complement table to write
- SeqRecord adds id, description, features

See course.bioinfo.se/session02.html, 'Biological objects'. NM_000518.5.

SciLifeLab

What does hbb[1:].translate() give?

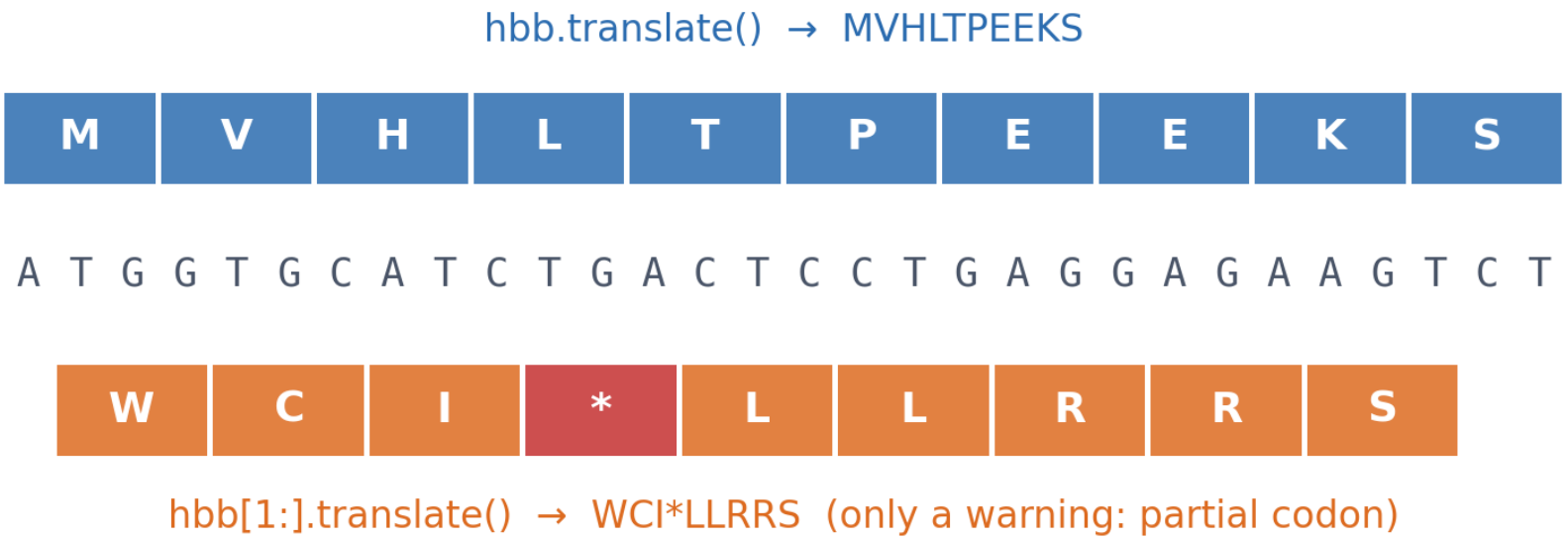
- **The same sequence, read from its second base**
- A: MVHLTPEEKS again
- B: an error
- C: a different peptide
- D: nothing



See course.bioinfo.se/session02.html, 'Biological objects', and its self-study questions.

SciLifeLab

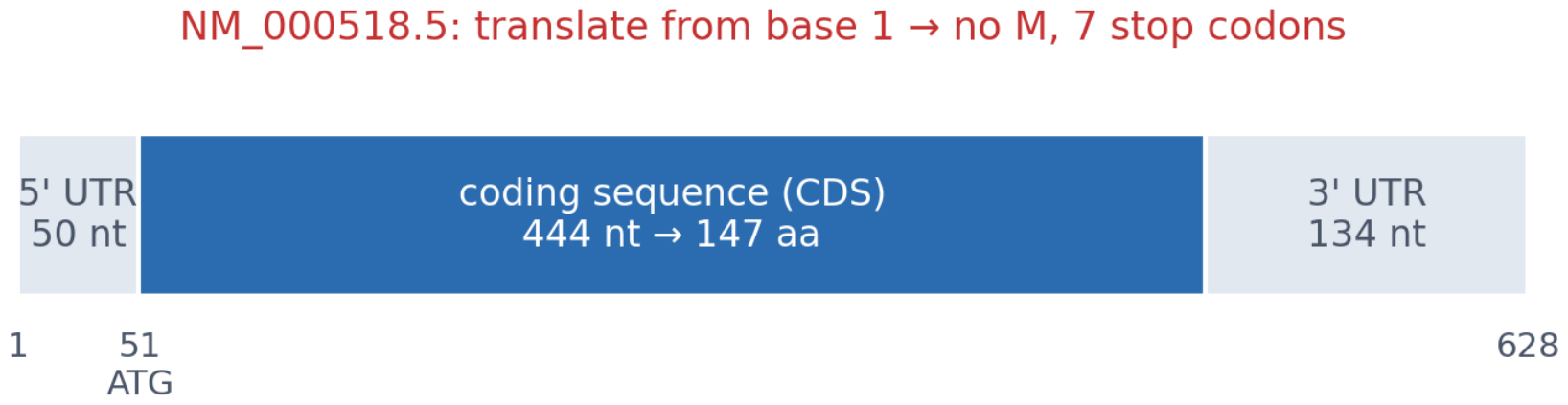
Biopython translates whatever you give it



Shifted by one base, every codon changes and a stop codon (*) appears. No error, only a warning: nothing stops you translating the wrong frame.

SciLifeLab

Translate the coding sequence, not the mRNA



The 628-nt HBB mRNA starts with a 50-nt untranslated region. The record's CDS feature gives the protein: cds.extract(record.seq).translate(to_stop=True) → 147 aa.

See course.bioinfo.se/session02.html, 'Biological objects'; NM_000518.5.

SciLifeLab

A class is a blueprint for objects

```
class Protein:
  def __init__(self, name, sequence):
    self.name = name
    self.sequence = sequence

  def length(self):
    return len(self.sequence)

  def weight(self): ...
```

the blueprint

```
beta = Protein('HBB', 'MVHLTPEEKS...')
```

the object beta

```
name     = 'HBB'
sequence = 147 residues
```

```
beta.length()   →  147
beta.weight()   →  15998 Da
```

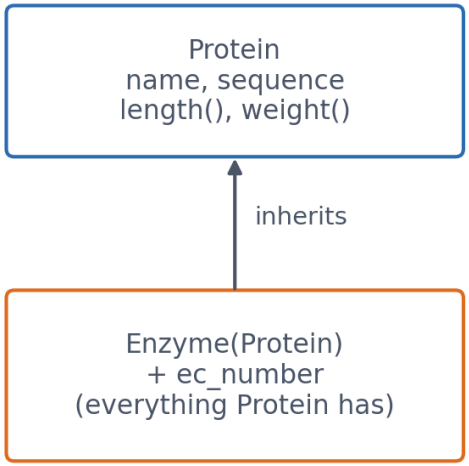
beta.length() is Protein.length(beta):
self is the object the method was called on

- **__init__ runs when an object is made**
- Attributes: the object's data
- Methods: functions on that data
- **self: this object**

See course.bioinfo.se/session02.html, 'Your own objects: classes'; course notebook, section 6. HBB: UniProt P68871.

SciLifeLab

Inheritance: an Enzyme is a Protein




```
class Enzyme(Protein):

  def __init__(self, name, sequence, ec):

    super().__init__(name, sequence)

    self.ec_number = ec
```

super().__init__ lets Protein set its attributes

```
lyz = Enzyme('LYZ', ..., '3.2.1.17')

lyz.weight()  →  14701 Da   (inherited)
```

forget super().__init__ → AttributeError, later

- **Inherits every method**
- Adds what is new
- **super().__init__() first**

See course.bioinfo.se/session02.html, 'Your own objects: classes'. Lysozyme: UniProt P61626, mature chain (130 aa).

SciLifeLab

A PyTorch model is a class

```
class TinyNet(nn.Module):
  def __init__(self):
    super().__init__()
    self.layer = nn.Linear(3, 1)

  def forward(self, x):
    return self.layer(x)
```

what it has:
layers and their weights
(attributes, set in __init__)

what it does:
input → output
(the forward method)



- **__init__: what it has**
- **forward: what it does**
- model(x) calls forward
- Every model from Session 8 on

See course.bioinfo.se/session02.html, 'Your own objects: classes'; course notebook, section 6.

SciLifeLab

A real dataset: mouse liver RNA-seq

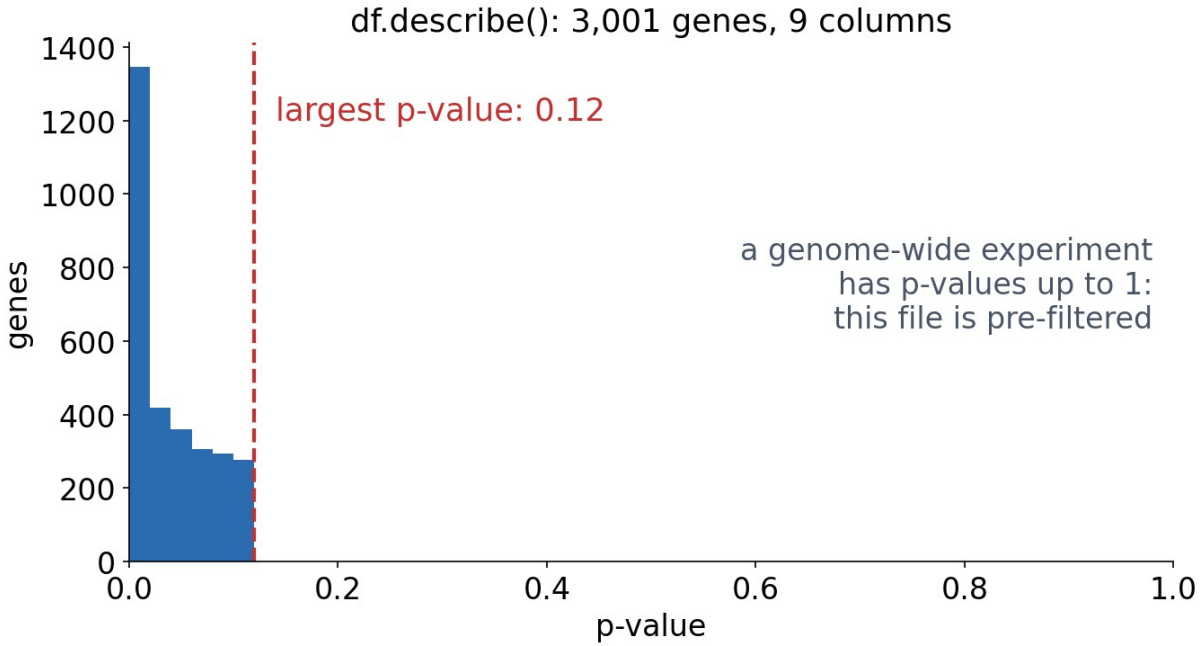
- **Becares et al. 2019, Cell Reports**
- LXRα-S196A mutant vs wild type, high-fat diet
- Already analysed with DESeq2
- One row per gene: baseMean, log2 FC, p, padj

3,001
genes

3 + 3
mice per group

SciLifeLab

Summaries tell you how the data were made

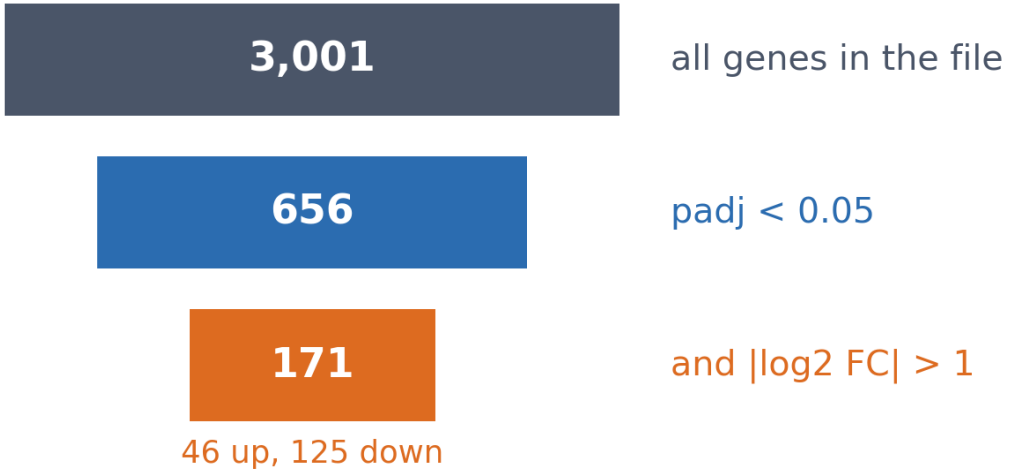


- **No p-value above 0.12**
- A pre-filtered list, not all genes
- Look before you compute

See course.bioinfo.se/session02.html, 'Exploring a biological dataset'.

SciLifeLab

Selecting and filtering

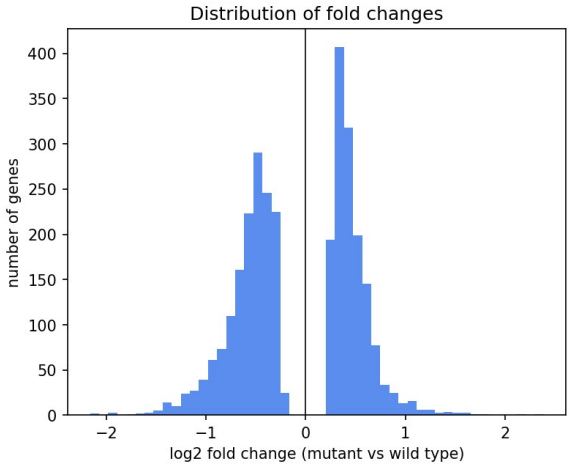


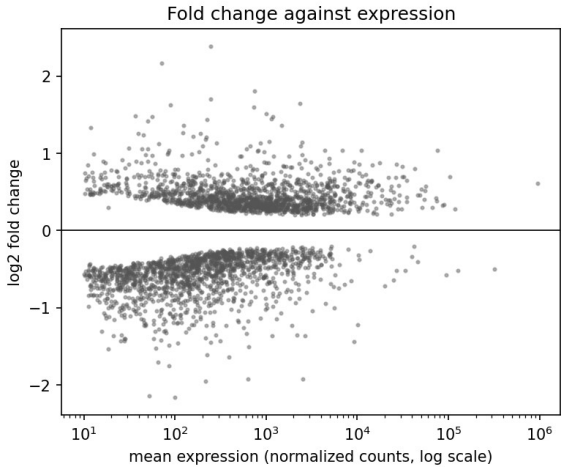
- **df[df['padj'] < 0.05]**
- Combine with &, not and
- Keep the parentheses

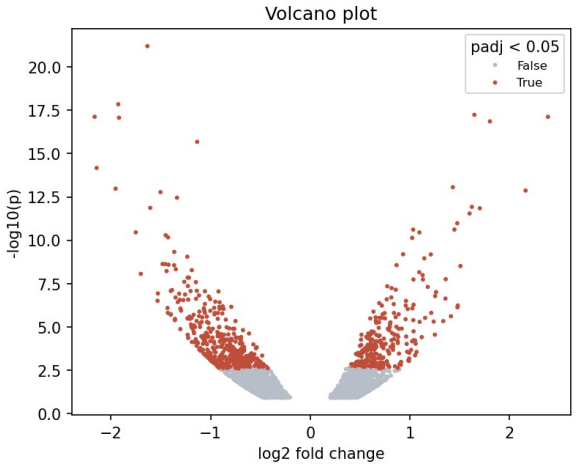
See course.bioinfo.se/session02.html, 'Exploring a biological dataset'.

SciLifeLab

Seeing the data







Histogram (one variable), scatter on a log axis (two variables), volcano plot coloured by significance with seaborn. The empty band around zero is the pre-filtering again.

Drawn in notebooks/session02-working-in-python.ipynb.

SciLifeLab

What do the two snippets return?

Looking for the most changed gene. log2_FoldChange: positive is up, negative is down.

```
df  (6 of 3 001 genes)

Gene     log2FC
Ces1f    +1.80
Syngr1   -2.16
Osbpl3   -1.63
Cyp2c69  +2.39
Mrgpre   -2.14
Elovl3   +2.16
```

\+ up
− down

```
A  df.sort_values("log2_FoldChange")
     .iloc[0]
```

```
B  df.loc[df["log2_FoldChange"]
     .abs().idxmax()]
```

A The same gene: both find the largest change

B A: most down; B: largest change, up or down

C A: most up; B: largest change, up or down

D A: most down; B: most up

Guess on your own

Scan the code, or go to
quiz.bioinfo.se/live

SciLifeLab

What do the two snippets return?

How did the room answer?

Halt the round when most have answered, then show the results.

Show the results ▸ quiz.bioinfo.se/live/report

SciLifeLab

What do the two snippets return?

Fix it, run it, discuss

Open the notebook: scan the code

Fix the line marked FIX, then run the cells

Did it come out as you guessed? Discuss with your neighbour

The notebook: https://colab.research.google.com/github/ElofssonLab/kb8029-book/blob/main/notebooks/session02-discussion-1.ipynb

SciLifeLab

What do the two snippets return?

Answer again

Same question, after the discussion.

quiz.bioinfo.se/live

SciLifeLab

What do the two snippets return?

Results, and who explains

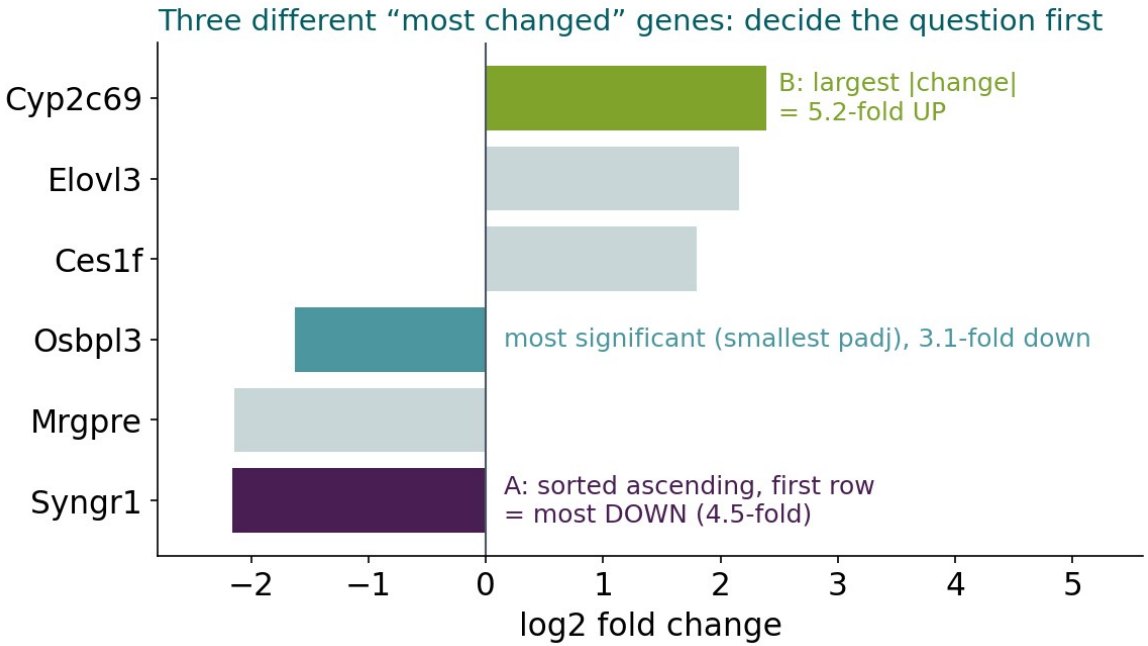
Both rounds side by side.

Draw two to explain: one of you says how you reasoned.

Show the results ▸ quiz.bioinfo.se/live/report

SciLifeLab

The answer: A finds the most down, B the largest change



sort_values() sorts ascending, so .iloc[0] is the most negative fold change. abs().idxmax() ignores the sign. The most significant gene is a third one, and the largest fold changes come mostly from lowly expressed genes. Decide the biological question first, then write the code.

SciLifeLab

BLOCK 1 DONE

Break

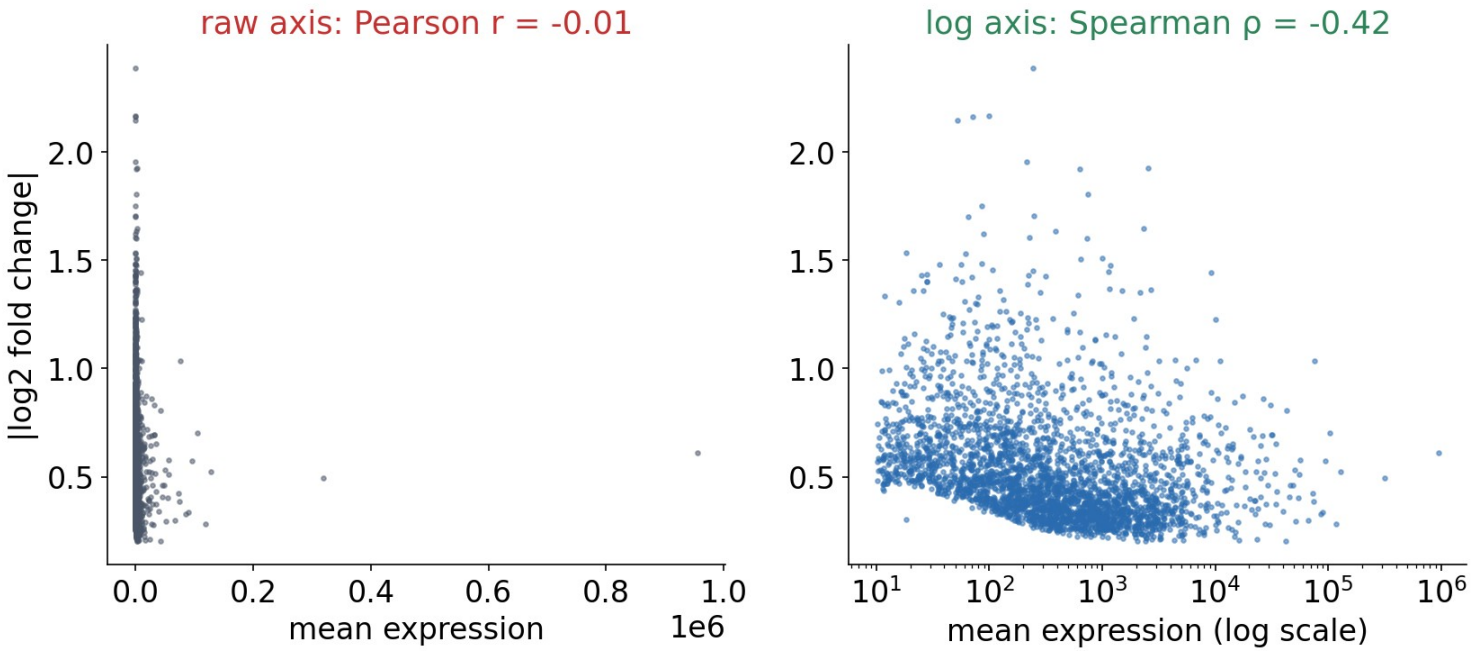
SciLifeLab

BLOCK 2 · ABOUT 30 MIN

Plot first, test second; data from somewhere; live coding

SciLifeLab

Plot first, test second



Is fold change related to expression level? Pearson on the raw numbers says no (r = −0.01, p = 0.43). The log-axis plot says yes, and so do Pearson on log10 expression (r = −0.30) and Spearman (ρ = −0.42, p = 2e-129).

See course.bioinfo.se/session02.html, 'Asking quantitative questions'.

SciLifeLab

Standard questions have standard functions

SciPy

stats.ttest_ind(..., equal_var=False)

- Session 1's Welch t-test
- t ≈ −11.9, p ≈ 0.0001
- Default equal_var=True: Student's

SciPy

cluster.hierarchy.linkage

- Merge the closest clusters
- Session 4's guide tree

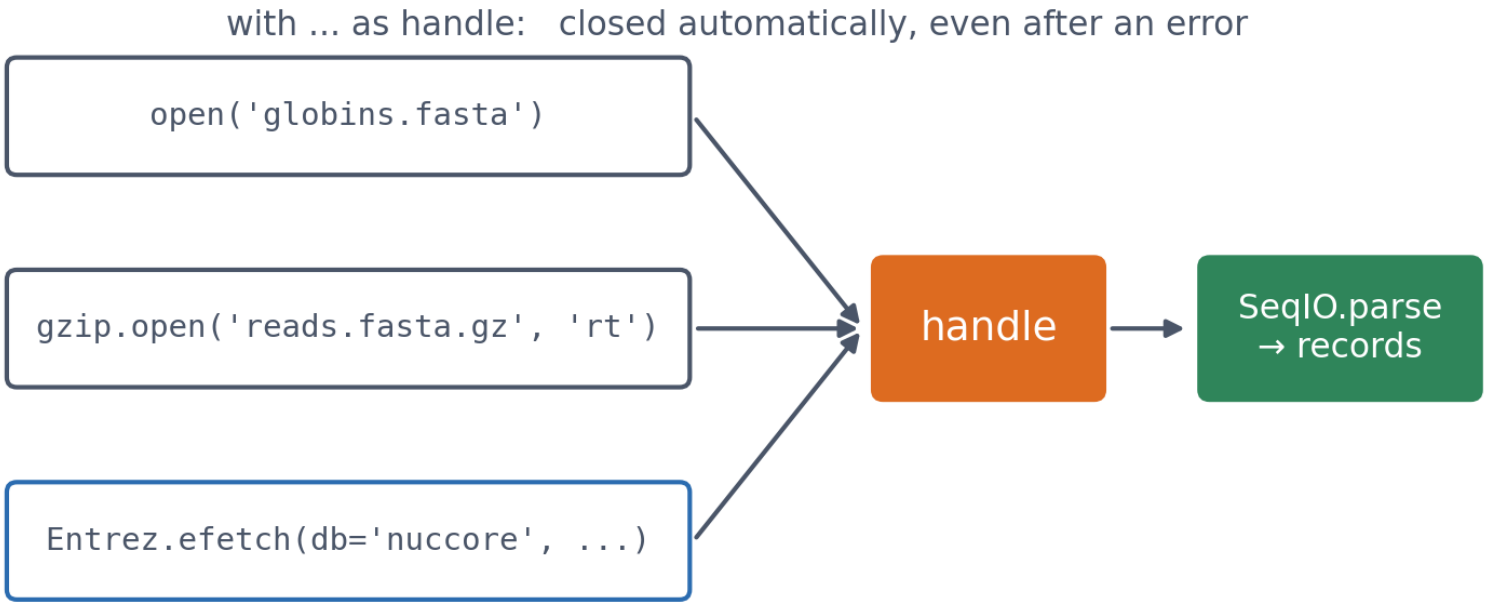
scikit-learn

LinearRegression, MLPClassifier, ...

- Sessions 7-9 build them by hand
- In practice: call the library

SciLifeLab

Files, compressed files and NCBI: one pattern



Every source gives you a handle; SeqIO.parse reads records from it. A .fasta.gz needs no unpacking, and zip/tar archives open with zipfile and tarfile.

See course.bioinfo.se/session02.html, 'Data coming from somewhere'.

SciLifeLab

Be polite to NCBI

- **Set Entrez.email**
- Fetch many records in one request
- Or pause between requests

~3 / s

requests allowed
without an API key

1

request for
50 accessions

See course.bioinfo.se/session02.html, 'Data coming from somewhere'.

SciLifeLab

LIVE CODING · ~15 MIN

Six globins, and three kinds of mistakes

SciLifeLab

Three kinds of mistakes

1 Python error	2 wrong analysis	3 nonsense biology
`KeyError: 'gc'`	`axes swapped`	`a 654-aa globin`
Python tells you	nothing fails	nothing fails, numbers look fine

AI assistants remove kind 1 well; kinds 2 and 3 need you

Watch for all three. Code running without an error does not mean the analysis is correct.

SciLifeLab

The task

Fetch

HBB, HBA1, HBA2, MB, NGB, CYGB

- One Entrez request

Tabulate

gene, length, gc_content

- A DataFrame

Plot

GC content vs mRNA length

- Length on the x-axis

Proteins

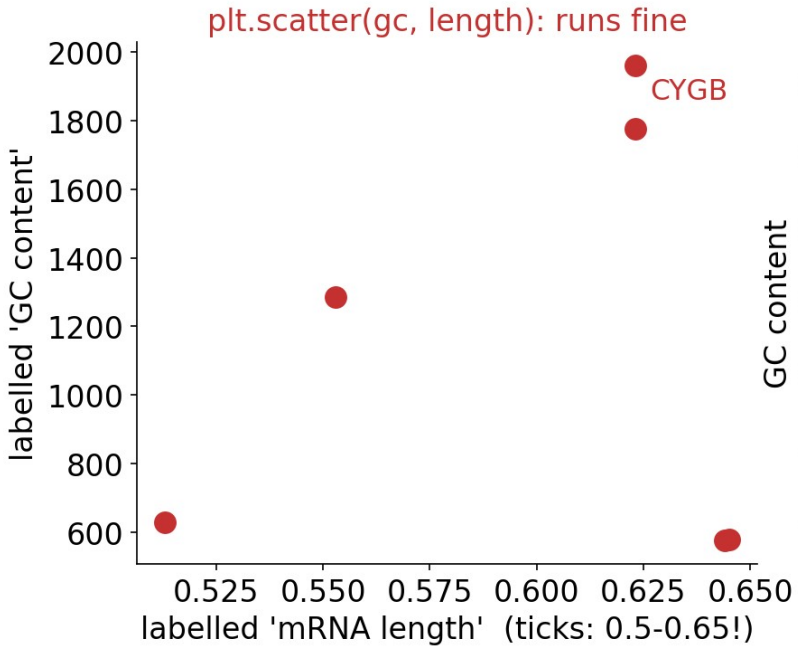
protein length

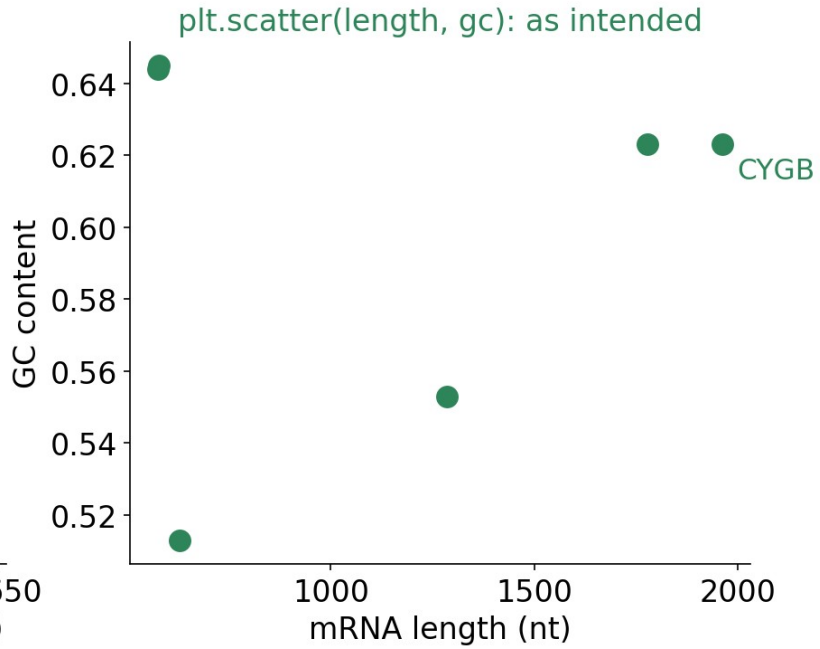
- For each gene

Script: slides/src/session02-live-coding.py.

SciLifeLab

It ran fine, and the labels lie

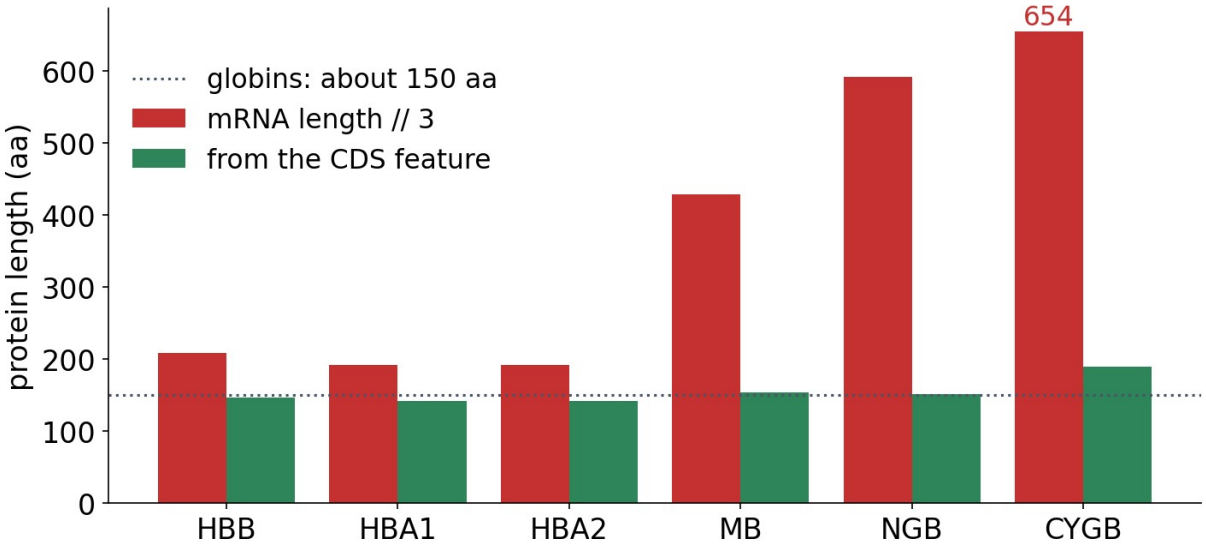




Find one point you know: CYGB has the longest mRNA, so it should be furthest right. Then read the ticks: 0.5-0.65 is GC content, not length.

Script: slides/src/session02-live-coding.py.

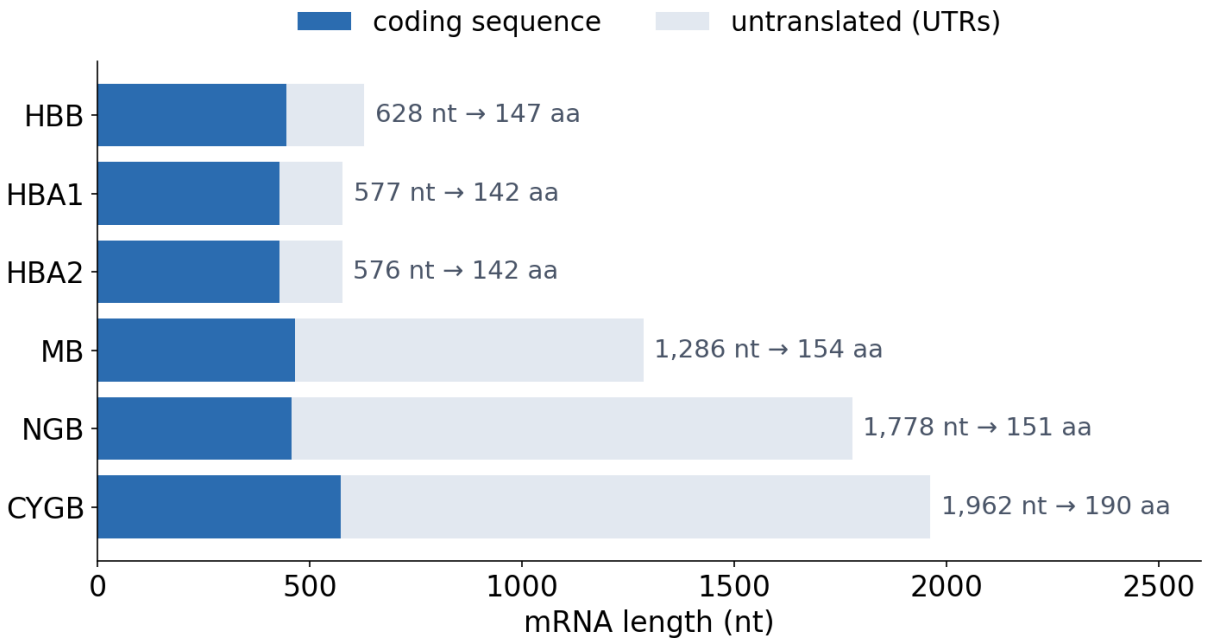
A 654-residue globin?



- **mRNA length // 3: runs fine**
- Globins are about 150 aa
- **Fix: the CDS feature**

Script: slides/src/session02-live-coding.py; book Table 2.3.

The difference is untranslated sequence



- **Proteins: 142-190 aa**
- **mRNAs: 576-1,962 nt**
- UTRs: 147-1,389 nt
- Length says little about the protein

See course.bioinfo.se/session02.html, 'Mini challenge', Table 2.3.

SciLifeLab

Which line contains the mistake?

The average length of the globins in a compressed FASTA file.

```
1  from Bio import SeqIO
2  import gzip
3  with gzip.open("globins.fasta.gz") as handle:
4      lengths = [len(r.seq) for r in
           SeqIO.parse(handle, "fasta")]
5  print(sum(lengths) / len(lengths))
```

A Line 3

B Line 4

C Line 5

D None: it runs and prints the average

Guess on your own

Scan the code, or go to
quiz.bioinfo.se/live

SciLifeLab

Which line contains the mistake?

How did the room answer?

Halt the round when most have answered, then show the results.

Show the results ▸ quiz.bioinfo.se/live/report

SciLifeLab

Which line contains the mistake?

Fix it, run it, discuss

Open the notebook: scan the code

Run the code: which line does Python point to?

Fix the mistake and run it again. Discuss with your neighbour

The notebook: https://colab.research.google.com/github/ElofssonLab/kb8029-book/blob/main/notebooks/session02-discussion-2.ipynb

SciLifeLab

Which line contains the mistake?

Answer again

Same question, after the discussion.

quiz.bioinfo.se/live

SciLifeLab

Which line contains the mistake?

Results, and who explains

Both rounds side by side.

Draw two to explain: one of you says how you reasoned.

Show the results ▸ quiz.bioinfo.se/live/report

To check it with code afterwards: discussion notebook 2 https://colab.research.google.com/github/ElofssonLab/kb8029-book/blob/main/notebooks/session02-discussion-2.ipynb

SciLifeLab

The answer: Python points at line 4, the mistake is on line 3

```
1   from Bio import SeqIO
2   import gzip
3   with gzip.open("globins.fasta.gz") as handle:
4       lengths = [len(r.seq) for r in SeqIO.parse(handle, "fasta")]
5   print(sum(lengths) / len(lengths))
```

```
Python:  ----> 4  lengths = [len(r.seq) for r in SeqIO.parse(handle, "fasta")]
         StreamModeError: Fasta files must be opened in text mode.
```

Fix: `with gzip.open("globins.fasta.gz", "rt") as handle:`

Python points at line 4, where it noticed the problem. The mistake is on line 3: gzip.open gives raw bytes by default; "rt" reads the file as text.

gzip.open() reads raw bytes unless you ask for text with "rt". Nothing goes wrong until SeqIO.parse tries to read text, on line 4. The line in an error message is where Python noticed the problem: read the last line first, then ask where the object it complains about came from.

SciLifeLab

BLOCK 2 DONE

Break

SciLifeLab

BLOCK 3 • ABOUT 30 MIN

When code does not work

SciLifeLab

A routine for getting unstuck



Experienced programmers follow the same five steps. The last one matters most when an AI assistant wrote the code.

See course.bioinfo.se/session02.html, 'When you don't know how to do something'.

SciLifeLab

Read the error from the bottom

```
KeyError      Traceback (most recent call last)
----> 1 df["log2FC"].mean()
  ...
KeyError: 'log2FC'
```

where

what went wrong

- **Last line: what went wrong**
- Arrow: where in your code
- KeyError on a DataFrame: no such column

→ df.columns shows the name is log2_FoldChange

SciLifeLab

Inspect instead of guessing

`type(x)`	what kind of object is this?
`x.shape, len(x), df.dtypes`	how big, and what is in it?
`df.columns`	which names exist?
`df.head()`	what does it actually look like?
`help(x)  or  x?`	what can it do?

- **Five tools answer most questions**
- Look at the object, not at your memory of it

See course.bioinfo.se/session02.html, 'When you don't know how to do something'.

SciLifeLab

The documentation is the authority for your version

```
from Bio.SeqUtils import GC        # old tutorials, AI assistants

ImportError: cannot import name 'GC' from 'Bio.SeqUtils'
```

```
from Bio.SeqUtils import gc_fraction   # Biopython 1.86 docs

gc_fraction(Seq('ATGGTGCATCTGACTCCTGAGGAGAAGTCT'))  →  0.5
```

- **Old tutorials still say GC**
- So do AI assistants
- The docs name gc_fraction

See course.bioinfo.se/session02.html. Biopython 1.86 when this deck was made.

SciLifeLab

Ask an AI assistant well, then verify

Give it

what a human helper would need

- The full error message
- df.head()
- Library versions
- What you expected, and what you tried

Then verify

it can invent or use outdated APIs

- Run it on a tiny known example
- Check shapes and ranges
- Look up unknown functions

See course.bioinfo.se/session02.html, 'How to use chatbots'. AI in graded work: the course rules on Canvas.

SciLifeLab

Test your assumptions in code

```
assert df["pvalue"].between(0, 1).all(), "p-values must lie between 0 and 1"

assert not df["Gene"].duplicated().any(), "one row per gene"

assert len(record.seq) == 628, "HBB mRNA should be 628 nt"
```

the notebook stops as soon as something you believe is false

An assert turns a belief about your data into a check that stops the notebook when it fails.

See course.bioinfo.se/session02.html, 'When you don't know how to do something'.

SciLifeLab

The notebook trap: stale state

what the page shows

```
[5]  counter = 0
```

```
[7]  counter = counter + 1
```

```
[9]  counter
```

out: 3

what the kernel remembers

counter = 3
(cell 7 was run three times)

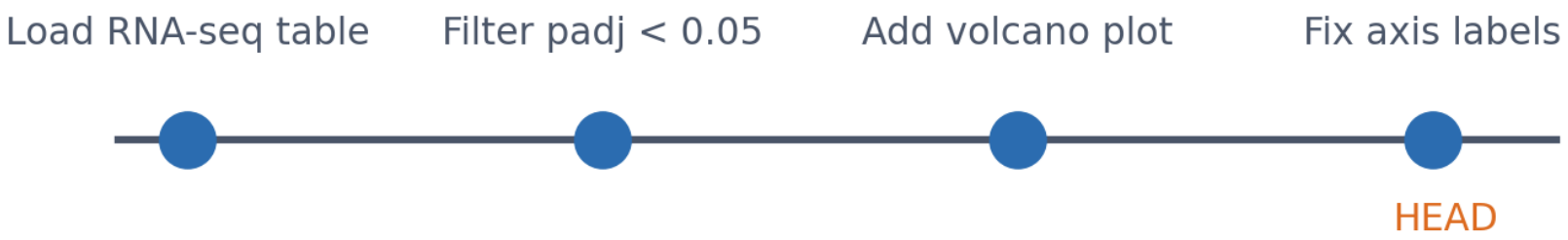
fix: Restart Kernel → Run All

- **Memory ≠ page**
- Out of order, re-run, edited
- **Before sharing: Restart → Run All**

See course.bioinfo.se/session02.html, 'Notebooks: what they are and why we use them'.

SciLifeLab

git: a history of snapshots

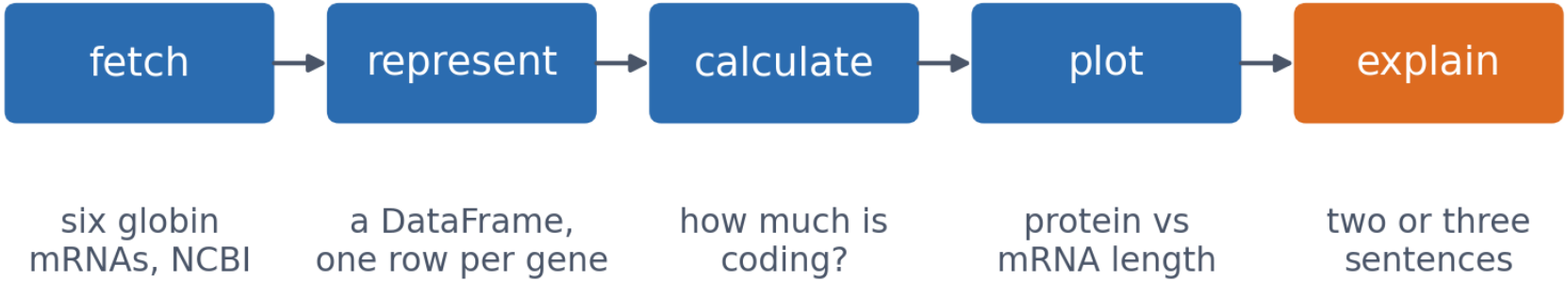


`git add · git commit -m "why" · git log · git pull / push`

Commit small, working steps, with messages that say why. Never commit big data, passwords or keys: list them in .gitignore.

SciLifeLab

Mini challenge: a small analysis, start to finish



The six globins again, now done properly: the result is the UTR figure from the live coding. The worked solution is in the course notebook.

See course.bioinfo.se/session02.html, 'Mini challenge'.

SciLifeLab

Fill in → adapt → solve



fill in	adapt	solve
complete todo(...) lines in working code	new questions; a cell with three bugs	twelve globins: does a longer mRNA mean a longer protein?

less help at each step · answer the Canvas quiz with your own output

One of the three bugs in the adapt part runs without an error and gives a wrong answer: a mistake of kind 2. Three optional extras follow (leptin with Entrez, seaborn, a Mann-Whitney comparison).

AI agents run whole analyses: who checks them?



an AI agent can run this whole loop without a human

1 · Python error	2 · wrong analysis	3 · nonsense biology
the agent sees it and fixes it	runs fine: nobody is told	numbers look fine: nobody is told

kinds 2 and 3 are still yours to catch

- Agents plan, write, run and fix code
- Kind 1 mistakes: fixed for you
- Kinds 2 and 3: still yours
- **Who is responsible for a result that no human looked at?**

Example: Biomni, a general-purpose biomedical AI agent, Huang et al. 2025, bioRxiv 10.1101/2025.05.30.656746.

SciLifeLab

What does x.grad show after a second backward()?

x = 3, y = x^2. After y.backward(), x.grad is 6. Then y2 = x^2 and y2.backward(), with no zeroing.

```
x = torch.tensor(3.0,
        requires_grad=True)

y = x**2
y.backward()
print(x.grad)   # 6

y2 = x**2       # no zeroing
y2.backward()
print(x.grad)   # ?
```

A 6: replaced

B 12: added

C 36: multiplied

D An error: only one backward() allowed

Guess on your own

Scan the code, or go to

quiz.bioinfo.se/live

SciLifeLab

What does x.grad show after a second backward()?

How did the room answer?

Halt the round when most have answered, then show the results.

Show the results ▸ quiz.bioinfo.se/live/report

SciLifeLab

What does x.grad show after a second backward()?

Fix it, run it, discuss

Open the notebook: scan the code

Fix the line marked FIX, then run the cells

Did it come out as you guessed? Discuss with your neighbour

The notebook: https://colab.research.google.com/github/ElofssonLab/kb8029-book/blob/main/notebooks/session02-discussion-3.ipynb

SciLifeLab

What does x.grad show after a second backward()?

Answer again

Same question, after the discussion.

quiz.bioinfo.se/live

SciLifeLab

What does x.grad show after a second backward()?

Results, and who explains

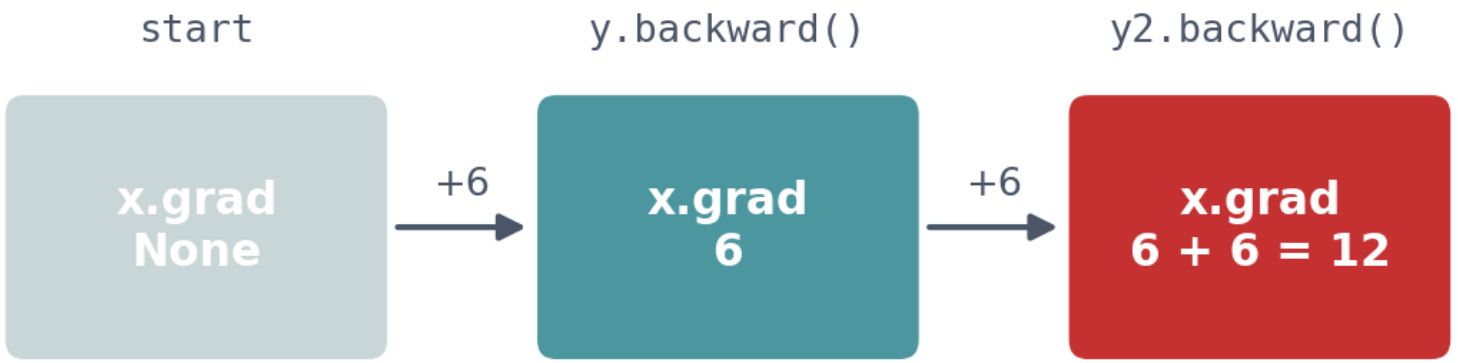
Both rounds side by side.

Draw two to explain: one of you says how you reasoned.

Show the results ▸ quiz.bioinfo.se/live/report

SciLifeLab

The answer: backward() adds to x.grad

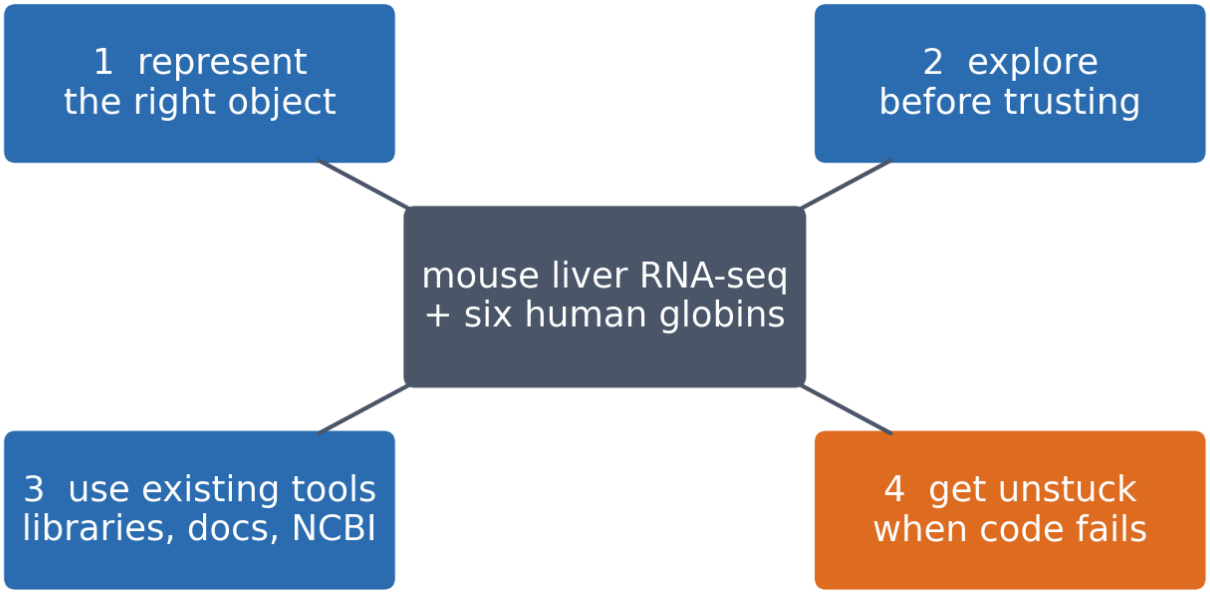


backward() ADDS to .grad. Training loops empty it first: optimizer.zero_grad()

The second backward() computes another 6 and adds it: 6 + 6 = 12. Code that runs and silently gives a wrong number: test your assumptions on a case you can work out by hand. Training loops empty the gradients at every step with optimizer.zero_grad() (Session 7).

SciLifeLab

Four ideas, and one rule



Represent with the right object, explore before trusting a number, use existing tools, and get unstuck with a routine. Code running without an error does not mean the analysis is correct.

See course.bioinfo.se/session02.html.

SciLifeLab

In up to three words: what was the hardest part of today?

Represent and explore	Questions and tools	Getting unstuck
Seq, DataFrame, class	plot first, SciPy, NCBI	errors, docs, git

Answer now

Scan the code, or go to
quiz.bioinfo.se/live

SciLifeLab

The hardest part of today

What the room found hardest

Show the results ▸ quiz.bioinfo.se/live/report

SciLifeLab

Session 3 next

Biological databases: identifiers, curated and predicted records
Following one protein, HBB, through five databases